21. 네트워크 설정 및 관리 분석

개요

Omni 네트워크는 다양한 환경(Devnet, Staging, Omega, Mainnet)에서 운영되며, 각 환경마다 고유한 설정과 관리 방식을 가지고 있습니다. 이 문서는 네트워크 설정, 배포, 업그레이드, 그리고 유지보수 과정을 상세히 분석합니다.

네트워크 환경 분류

네트워크 타입 정의

```
const (

// Simnet은 개별 바이너리 테스트를 위한 시뮬레이션 네트워크

Simnet ID = "simnet"  // 단일 바이너리, 모킹된 클라이언트 (네트워킹 없음)



// Devnet은 가장 기본적인 단일 머신 배포

Devnet ID = "devnet"



// Staging은 Omni 팀의 내부 스테이징 네트워크

Staging ID = "staging"



// Omega는 Omni 공개 테스트넷

Omega ID = "omega"
```

```
// Mainnet은 Omni 공개 메인넷
Mainnet ID = "mainnet"
)
```

네트워크 특성 분류

임시 네트워크 (Ephemeral):

- `Simnet`, `Devnet`, `Staging`
- 단기간 사용, 빠른 반복 개발
- 설정 변경 및 재배포 용이

보호 네트워크 (Protected):

- `Omega`, `Mainnet`
- 장기간 운영, 안정성 중시
- 설정 변경 시 신중한 검토 필요

네트워크 설정 구조

정적 설정 (Static Configuration)

```
type Static struct {
Network ID
OmniExecutionChainID  uint64
AVSContractAddress common.Address
```

```
TokenAddress common.Address

L1BridgeAddress common.Address

Portals []Deployment

MaxValidators  uint32

UnbondingTime time.Duration

ConsensusGenesisJSON []byte

ConsensusSeedTXT []byte

ConsensusArchiveTXT []byte

ExecutionGenesisJSON []byte

ExecutionSeedTXT []byte

OmniScanBaseURL  string

}
```

네트워크별 정적 설정

Mainnet 설정:

```
Mainnet: {

Network: Mainnet,

AVSContractAddress: mainnetAVS,

OmniExecutionChainID: evmchain.IDOmniMainnet,

MaxValidators: 30,

UnbondingTime: time.Hour * 24 * 7 * 3, // 3주

TokenAddress: mainnetToken,
```

```
L1BridgeAddress: mainnetBridge,

Portals: []Deployment{

{ChainID: evmchain.IDEthereum, Address: mainnetPortal, DeployHeight: 21029795},

{ChainID: evmchain.IDArbitrumOne, Address: mainnetPortal, DeployHeight: 26688962

{ChainID: evmchain.IDOptimism, Address: mainnetPortal, DeployHeight: 127052933},

{ChainID: evmchain.IDBase, Address: mainnetPortal, DeployHeight: 21457647},

{ChainID: evmchain.IDOmniMainnet, Address: mainnetPortal, DeployHeight: 4407},

},

// ... 기타 설정

}
```

Staging 설정:

```
Staging: {

Network: Staging,

OmniExecutionChainID: evmchain.IDOmniStaging,

MaxValidators: 30,

UnbondingTime: 5 * time.Second, // 빠른 테스트를 위한 짧은 시간

TokenAddress: stagingToken,

// ... 기타 설정

}
```

매니페스트 기반 네트워크 정의

매니페스트 구조

```
type Manifest struct {
e2e.Manifest

Network netconf.ID `toml:"network"`

// 체인 설정
AnvilChains []string  `toml:"anvil_chains"`  // 로컬 테스트 체인
PublicChains []string  `toml:"public_chains"`  // 공개 체인 연결
MultiOmniEVMs  bool  `toml:"multi_omni_evms"`  // 다중 Omni EVM 배포

// 테스트 설정
PingPongN  uint64  `toml:"pingpong_n"`
AllE2ETests  bool  `toml:"all_e2e_tests"`

// 키 관리
Keys  map[string]map[key.Type]string  `toml:"keys"`

// 버전 고정
PinnedHaloTag  string  `toml:"pinned_halo_tag"`
PinnedMonitorTag  string  `toml:"pinned_monitor_tag"`
PinnedRelayerTag  string  `toml:"pinned_relayer_tag"`
```

```
PinnedSolverTag  string  `toml:"pinned_solver_tag"`


// 업그레이드 설정
EphemeralGenesis  string  `toml:"ephemeral_genesis"`
FeatureFlags feature.Flags `toml:"feature_flags"`
}
```

네트워크별 매니페스트 예시

Devnet1 매니페스트:

```
# Devnet1은 간단한 다중 검증자 개발 네트워크
network = "devnet"
anvil_chains = ["mock_l1", "mock_l2"]


multi_omni_evms = true
prometheus = true
all_e2e_tests = true
ephemeral_genesis = "2_magellan"


[node.validator01]
[node.validator02]


[node.fullnode01]
```

```
mode="archive"
```

Mainnet 매니페스트:

```
network = "mainnet"

public_chains = ["ethereum","arbitrum_one","base","optimism"]


multi_omni_evms = true

prometheus = true


pinned_halo_tag = "0609013"

pinned_relayer_tag = "07c4ac7"

pinned_monitor_tag = "ee06e15"

pinned_solver_tag = "902fc10"


[node.validator01]

[node.validator02]

[node.validator03]

[node.validator04]


[node.seed01]

mode = "seed"

[node.seed02]

mode = "seed"
```

```
[node.archive01]

mode = "archive"

[node.archive02]

mode = "archive"

[node.archive03]

mode = "archive"
```

시드 노드 및 부트스트랩 관리

시드 노드 구성

Mainnet 합의 시드:

```
d0a9109801c7f78759a20765c30b00d06a6c2a8f@seed01.mainnet.omni.network:26656

c84e6a5e3c29f1b59b225613be056e82ab5663c7@seed02.mainnet.omni.network:26656
```

Mainnet 실행 시드:

```
enode://fd3ca08c7f37658add8ac37f1671b4dc0e7bf849bf60c050ef9ce5fa28debf344fba49ad

enode://d5e463390f628349657bd5828d491c87e68d346b60babdc13d3bfb50f7c66249aebbd368
```

아카이브 노드 구성

Mainnet 아카이브 노드:

```
eae99c1f0faaba9ad96c943f17de350566b028dd@archive01.mainnet.omni.network:26656

95f7a9277e3ffb37b81e0653f23ce8c57f9dfbc0@archive02.mainnet.omni.network:26656

a0451a082279bdb8971a964cb12fa3c8d1ad8be4@archive03.mainnet.omni.network:26656
```

네트워크 연결 확인 스크립트

```
#!/usr/bin/env bash

# scripts/check_network.sh


NETWORK="${1}"


# 합의 시드 연결 확인

for SEED in $(cat "${SCRIPT_DIR}/../lib/netconf/${NETWORK}/consensus-seeds.tx

HOSTPORT=$(echo "${SEED}" | cut -d@ -f2)

HOST=$(echo "${HOSTPORT}" | cut -d: -f1)

PORT=$(echo "${HOSTPORT}" | cut -d: -f2)

echo "Checking connection to consensus seed: ${HOST}:${PORT}"

nc -z -v -w 1 "${HOST}" "${PORT}"

done


# 실행 시드 연결 확인

for SEED in $(cat "${SCRIPT_DIR}/../lib/netconf/${NETWORK}/execution-seeds.tx

HOSTPORT=$(echo "${SEED}" | cut -d@ -f2)
```

```
HOST=$(echo "${HOSTPORT}" | cut -d: -f1)
PORT=$(echo "${HOSTPORT}" | cut -d: -f2)
echo "Checking connection to execution seed: ${HOST}:${PORT}"
nc -z -v -w 1 "${HOST}" "${PORT}" # TCP 연결
nc -z -v -u -w 1 "${HOST}" "${PORT}" # UDP 연결
done
```

배포 인프라 관리

Docker 기반 배포 (Devnet)

Docker Compose 설정:

```
# 동적 생성되는 docker-compose.yaml 예시
version: '3.8'
services:
validator01:
image: omniops/halovisor:main
container_name: validator01
volumes:
- ./validator01:/halo
ports:
- "5701:26657"
networks:
```

```
omni:
ipv4_address: 172.16.0.10


validator01_evm:
image: ethereum/client-go:stable
container_name: validator01_evm
volumes:
- ./validator01_evm:/geth
ports:
- "8000:8545"
networks:
omni:
ipv4_address: 172.16.0.11
```

VM Compose 기반 배포 (Staging/Production)

VM별 서비스 분산:

```
func groupByVM(instances map[string]e2e.InstanceData) map[string]map[string]bo
resp := make(map[string]map[string]bool) // map[vm_ip]map[service_name]true
for serviceName, instance := range instances {
ip := instance.IPAddress.String()
m, ok := resp[ip]
if !ok {
```

```
m = make(map[string]bool)
}
m[serviceName] = true
resp[ip] = m
}
return  resp
}
```

VM 배포 과정:

1. **아티팩트 복사**: 로컬에서 VM으로 설정 파일 전송
2. **Docker Compose 생성**: VM별 맞춤형 compose 파일 생성
3. **서비스 시작**: `docker compose up -d` 로 서비스 시작
4. **백업**: GCP Storage에 설정 백업

네트워크 업그레이드 관리

Halovisor를 통한 무중단 업그레이드

업그레이드 바이너리 관리:

```
# Halovisor Dockerfile
FROM omniops/halo:${HALO_VERSION_0_GENESIS} AS build-0-genesis
FROM omniops/halo:${HALO_VERSION_1_ULUWATU} AS build-1-uluwatu
FROM omniops/halo:${HALO_VERSION_2_MAGELLAN} AS build-2-magellan
FROM omniops/halo:${HALO_VERSION_3_DRAKE} AS build-3-drake
```

```
FROM omniops/halo:${HALO_VERSION_N_LATEST} AS build-4-earhart


# 런타임 스테이지

FROM scratch AS runtime

COPY --from=build-cosmovisor /ko-app/cosmovisor /usr/local/bin/cosmovisor

COPY --from=build-0-genesis /app /halovisor/genesis/bin/halo

COPY --from=build-1-uluwatu /app /halovisor/upgrades/1_uluwatu/bin/halo

COPY --from=build-2-magellan /app /halovisor/upgrades/2_magellan/bin/halo

COPY --from=build-3-drake /app /halovisor/upgrades/3_drake/bin/halo

COPY --from=build-4-earhart /app /halovisor/upgrades/4_earhart/bin/halo
```

Cosmovisor 환경 변수:

```
ENV DAEMON_HOME=/halo

ENV DAEMON_NAME=halo

ENV DAEMON_ALLOW_DOWNLOAD_BINARIES=false

ENV DAEMON_RESTART_AFTER_UPGRADE=true

ENV UNSAFE_SKIP_BACKUP=true

ENV COSMOVISOR_TIMEFORMAT_LOGS=rfc3339nano

ENV COSMOVISOR_CUSTOM_ROOT=/halovisor

ENV COSMOVISOR_CUSTOM_CURRENT_LINK=/halo/halovisor-current

ENV COSMOVISOR_CUSTOM_CURRENT_UPGRADEINFO=/halo/halovisor-upgradeinfo.json
```

업그레이드 프로세스

1. **새 버전 빌드**: GoReleaser로 새 Docker 이미지 생성
2. **Halovisor 업데이트**: 새 바이너리를 포함한 Halovisor 이미지 빌드
3. **점진적 배포**:

- 테스트넷에서 먼저 검증
- 메인넷에 단계적 롤아웃

4. **업그레이드 신호**: 거버넌스 제안을 통한 업그레이드 활성화
5. **자동 전환**: Cosmovisor가 지정된 블록 높이에서 자동 전환

포털 컨트랙트 배포 관리

포털 배포 프로세스

```
func (m *manager) deployIfNeeded(ctx context.Context, chain types.EVMChain, back

isDeployed, addr, err := portal.IsDeployed(ctx, m.network, backend)

if  err != nil {

return common.Address{}, 0, errors.Wrap(err, "is deployed", "chain", chain)

}


staticDeploy, hasStatic := m.network.Static().PortalDeployment(chain.ChainID)


// 임시 네트워크의 경우 포털이 이미 배포되어 있으면 오류

if  isDeployed && m.network.IsEphemeral() {

return common.Address{}, 0, errors.New("ephemeral portal already deployed")
```

```
}

// 정적 배포 설정이 있으면 사용
if hasStatic {
return staticDeploy.Address, staticDeploy.DeployHeight, nil
}

// 포털 배포
addr, receipt, err := portal.Deploy(ctx, m.network, backend, feeOracle, valSetID
if err != nil {
return common.Address{}, 0, errors.Wrap(err, "deploy portal")
}

return addr, receipt.BlockNumber.Uint64(), nil
}
```

포털 레지스트리 관리

동적 네트워크 구성:

```
func AwaitOnExecutionChain(ctx context.Context, netID ID, portalRegistry *bindi
for {
portals, err := portalRegistry.List(&bind.CallOpts{Context: ctx})
```

```
if  err != nil {
log.Warn(ctx, "Failed fetching network from execution registry (will retry)", er
backoff()
continue
}

network, err := networkFromPortals(ctx, netID, portals)
if  err != nil {
return Network{}, err
}

if !containsAll(network, expected) {
log.Info(ctx, "Execution registry doesn't contain all expected chains (will retr
backoff()
continue
}

return  network, nil
}
}
```

모니터링 및 관찰성

Prometheus 메트릭 수집

네트워크별 메트릭 설정:

```
func  writeMonitorConfig(ctx context.Context, def Definition, logCfg log.Config,

cfg := monapp.DefaultConfig()

cfg.Network = def.Testnet.Network

cfg.HaloCometURL = archiveNode.AddressRPC()

cfg.SolverAddress = def.Testnet.SolverInternalAddr

cfg.HaloGRPCURL = haloGRPCAddress(archiveNode)

cfg.RPCEndpoints = endpoints

cfg.XFeeMngr.RPCEndpoints = xfeemngrEndpoints

return  monapp.WriteConfigTOML(cfg, logCfg, filepath.Join(confRoot, configFile))

}
```

네트워크 상태 추적

검증자 상태 모니터링:

- 블록 생성 상태
- 합의 참여율
- 슬래싱 이벤트
- 네트워크 지연시간

크로스체인 메시지 추적:

- 메시지 전송 성공률

- 릴레이 지연시간
- 실패한 메시지 분석

보안 및 키 관리

키 타입별 관리

```
type Type string

const (
Validator Type = "validator"  // 검증자 서명 키
P2PConsensus Type = "p2p_consensus"  // 합의 P2P 키
P2PExecution Type = "p2p_execution"  // 실행 P2P 키
)
```

네트워크별 키 관리 전략

Devnet:

- 결정론적 비보안 키 생성
- 개발 편의성 우선

Staging/Testnet:

- GCP에서 키 다운로드
- 제한된 보안 수준

Mainnet:

- Fireblocks 하드웨어 보안 모듈

- 다중 서명 및 승인 프로세스

Fireblocks 통합

```
func NewFireblocksClient(cfg DefinitionConfig, networkID netconf.ID, commandNam
key, err := fireblocks.LoadKey(cfg.FireKeyPath)
if err != nil {
return fireblocks.Client{}, errors.Wrap(err, "load fireblocks key")
}

opts := []fireblocks.Option{
fireblocks.WithSignNote(fmt.Sprintf("omni e2e %s %s", commandName, networkID)),
fireblocks.WithQueryInterval(5 * time.Second),
}

return fireblocks.New(networkID, cfg.FireAPIKey, key, opts...)
}
```

네트워크 유지보수 절차

정기 유지보수

1. **노드 상태 점검**:

- 디스크 사용량 모니터링

- 메모리 및 CPU 사용률 확인
- 네트워크 연결 상태 검증

2. **백업 관리**:

- 설정 파일 백업 (GCP Storage)
- 체인 데이터 스냅샷
- 키 파일 안전 보관

3. **성능 최적화**:

- 불필요한 Docker 이미지 정리
- 로그 파일 로테이션
- 데이터베이스 압축

긴급 대응 절차

네트워크 중단 시:

1. 문제 노드 식별
2. 백업 노드로 트래픽 전환
3. 문제 원인 분석 및 수정
4. 점진적 서비스 복구

보안 사고 시:

1. 영향받은 노드 격리
2. 키 순환 (필요시)
3. 네트워크 상태 검증
4. 사후 분석 및 개선

자동화 도구

Makefile 명령어

```
# 개발 네트워크 배포
devnet-deploy:
@go run github.com/omni-network/omni/e2e -f e2e/manifests/$(MANIFEST).toml deplo

# 네트워크 정리
devnet-clean:
@go run github.com/omni-network/omni/e2e -f e2e/manifests/$(MANIFEST).toml clean

# E2E 테스트 실행
e2e-run:
@go run github.com/omni-network/omni/e2e -f e2e/manifests/$(MANIFEST).toml $(RUN
```

CI/CD 파이프라인

자동 배포 플로우:

1. 코드 변경 감지
2. 테스트 실행
3. Docker 이미지 빌드
4. 스테이징 환경 배포
5. 검증 테스트 실행
6. 프로덕션 배포 승인

7. 점진적 롤아웃

결론

Omni의 네트워크 설정 및 관리 시스템은 다음과 같은 특징을 가집니다:

1. **계층화된 네트워크 구조**: 개발부터 프로덕션까지 단계적 환경 제공
2. **매니페스트 기반 설정**: 선언적 네트워크 구성 관리
3. **무중단 업그레이드**: Halovisor와 Cosmovisor를 통한 안전한 업그레이드
4. **자동화된 배포**: Docker와 VM Compose를 통한 일관된 배포
5. **강화된 보안**: 네트워크별 차등화된 키 관리 전략
6. **포괄적 모니터링**: 실시간 네트워크 상태 추적 및 알림

이러한 시스템을 통해 Omni는 안정적이고 확장 가능한 크로스체인 네트워크 운영을 실현하고 있습니다.